

LAB 4: LoRa - I

La presente evaluación será recibida sólo si se ha cumplido con los checkpoints de todas las preguntas en el **LABORATORIO** con el docente del curso.

La entrega debe constar de:

1. **Introducción** (incluir párrafo introductorio):
 1. **Marco Teórico** (definiciones **con referencias**)
 2. **Estado del Arte** (trabajos de investigación, artículos, tesis, páginas **con referencias**).
2. **Metodología** (explicación de componentes e implementos utilizados y diagramas de flujo/bloques).
3. **Desarrollo de la experiencia** (incluye discusión)
4. **Conclusiones** (una por cada experiencia realizada **como mínimo**)
5. **Referencias** (formato IEEE)

Consideraciones:

- Subir únicamente el PDF del informe a Canvas, usar un link para el repositorio, incluir fotos de paso a paso de cada implementación y un vídeo mostrando el funcionamiento de las experiencias. Sin embargo, para simulaciones, considere la placa Arduino UNO disponible en Tinkercad.
- Considere una placa de Arduino MEGA2560 tal cual existe en el laboratorio.
- Solo un miembro del grupo sube el informe.
- No olvidar colocar los nombres de todos los integrantes.

ÍNDICE

1. Introducción a Comunicación LoRa (Comandos AT)	3
1.1. Instrucciones	3
2. Carga de Código de Prueba a RAK3172S	5
2.1. Instrucciones	5
2.2. Se pide	5
3. Implementación de sistema de comunicación P2P con LoRa	6
3.1. Instrucciones:	6
4. Implementación con Arduino y sensor	7
5. LoRa	9
5.1. Características	9
6. Dispositivos RAK	9
7. Conexión de dispositivos RAK con Conversor TTL	10
8. RAK SERIAL PORT TOOL	11
9. RAK WisToolBox	12
10. Paquete de soporte de la placa RAK3272S RUI3 en Arduino IDE	14
11. Comandos AT (RUI v3)	18
11.1. RUI3 Formato de comandos AT	18
12. Código de Ejemplo Arduino + RAK	21

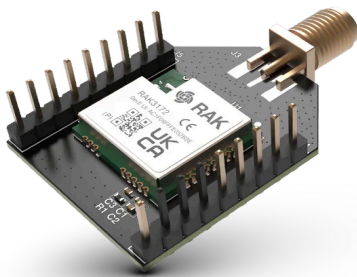
LoRa - I

Consideraciones iniciales:

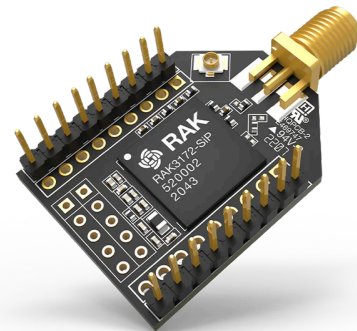
- **IMPORTANTE: NIVEL DE ALIMENTACIÓN 3.3V**
- **IMPORTANTE: NUNCA ENERGIZAR SIN LA ANTENA.**

Instrucciones Iniciales

Como parte de la experiencia de laboratorio se cuentan con los dispositivos:



RAK3172 Breakout Board



RAK3272-SiP Breakout Board

Los cuales son de la empresa RAKWireless. Estos dispositivos trabajan con un estandar denominado **RUI3 (Rak United Interface)**, el cual emplea comandos AT para realizar la programación de los mismos.

Información complementaria de cada dispositivo se encuentra através de los siguientes enlaces:

[RAK3172 Datasheet](#)

[RAK3272-SiP Datasheet](#)

1. Introducción a Comunicación LoRa (Comandos AT)

IMPORTANTE: NIVEL DE ALIMENTACIÓN 3.3V Y NUNCA ENERGIZAR SIN LA ANTENA.

Información adicional de Comandos AT: [RUI3 - AT Command Manual](#)

1.1. Instrucciones

- Conectar mediante el conversor serial ambos dispositivos.
 - Nota Importante: Los dispositivos RAK reciben instrucciones de comandos a través de los puertos **UART2 (RX2 y TX2)**.

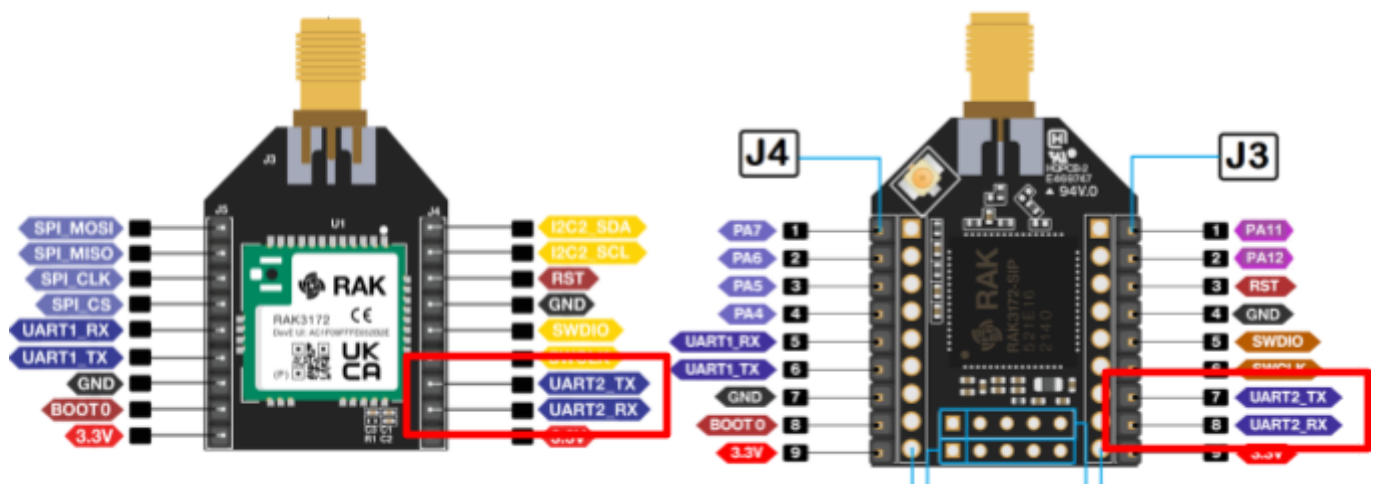


Figura 1. Puertos de comunicación UART.

Nota: Se recomienda usar el RAK Serial Port Tool o el Wistoolbox

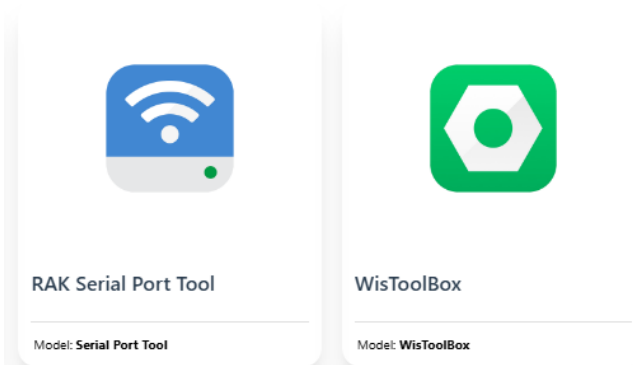


Figura 3. Herramientas de programación directa de dispositivos RAK.

[Enlace a RAK Tools](#)

[WisToolBox](#)

- Realizar la ejecución de los siguientes comandos a través de comunicación serial:
 - AT+VER=?
 - ATZ
- Mostrar el resultado de ejecución de código en su dispositivo (adjuntar captura en su informe).
- **Antes de realizar la primera conexión al ordenador, consultar al docente.**

Checkpoint 01

2. Carga de Código de Prueba a RAK3172S

2.1. Instrucciones

- Seguir la guía de uso de **Paquete de soporte de la placa RAK3272S RUI3 en Arduino IDE**, mostrada en los **Anexos**.
- Compilar el ejemplo RAK3172-E.ino en su placa RAK3272S.

```
RAK3172-E.ino
1
2 extern const char *sw_version;
3
4 void setup()
5 {
6     uint32_t baudrate = Serial.getBaudrate();
7     Serial.begin(baudrate);
8     Serial.println("RAKwireless RAK3172-E");
9     Serial.println("-----");
10    Serial.printf("Version: %s\r\n", sw_version);
11 }
12
13 void loop()
14 {
15     /* Destroy this busy loop and use timer to do what you want instead,
16      * so that the system thread can auto enter low power mode by api.system.lpm.set(1); */
17     api.system.scheduler.task.destroy();
18 }
19
```

2.2. Se pide

- Mostrar los resultados de ejecución al docente.

```
Output Serial Monitor
[S] TRANSMISSION: block 47 (seq=48) sent
[S] TRANSMISSION: block 48 (seq=49) sent
[S] TRANSMISSION: block 49 (seq=50) sent
[S] TRANSMISSION: block 50 (seq=51) sent
[S] TRANSMISSION: block 51 (seq=52) sent
[S] TRANSMISSION: block 52 (seq=53) sent
[S] TRANSMISSION: block 53 (seq=54) sent
[S] TRANSMISSION: block 54 (seq=55) sent
[S] TRANSMISSION: block 55 (seq=56) sent
```

```
Entering boot mode
Device is in boot mode
Upgrade Complete
```

RAKwireless RAK3172-E

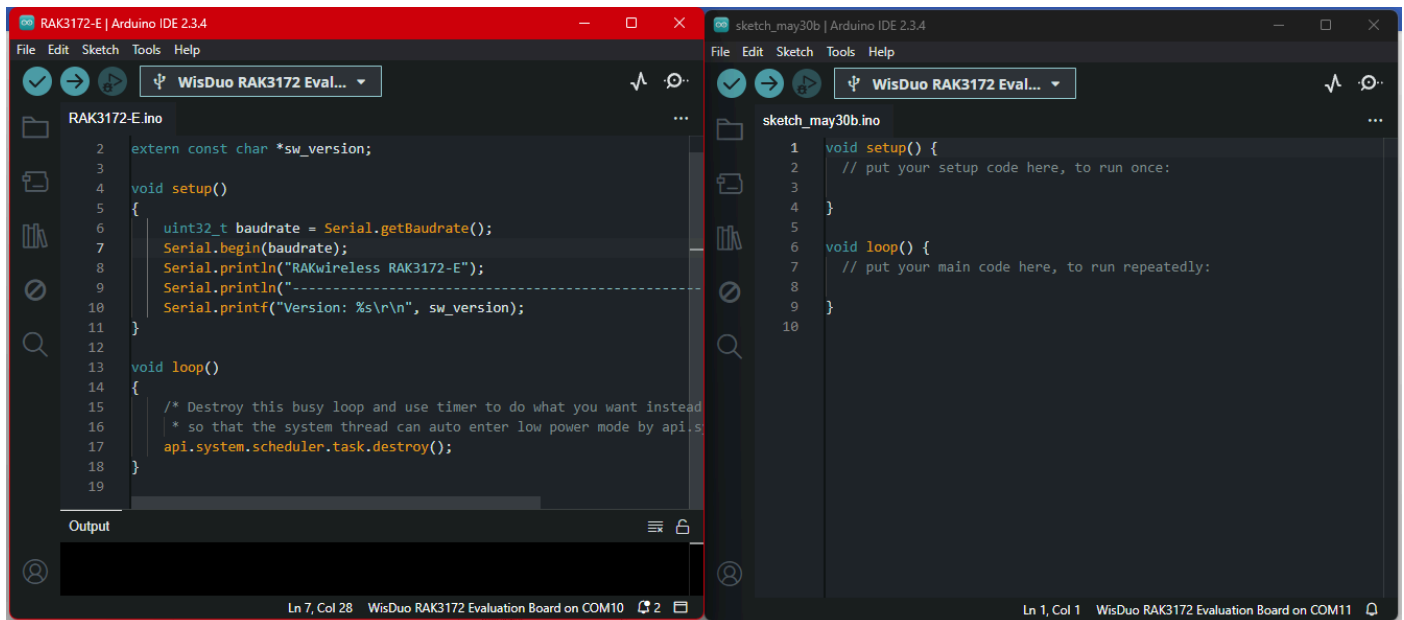
Version: RUI_4.2.1_RAK3172-E

Current Work Mode: LoRa P2P.

Checkpoint 02

3. Implementación de sistema de comunicación P2P con LoRa

- En esta parte solicitar al docente un segundo RAK y un segundo conversor USB a TTL
- Implemente un sistema de transmisión LoRa P2P utilizando los dos dispositivos RAK:
 - RAK3272S
- Primero realice la conexión UART con el dispositivo RAK usando el IDE de Arduino.
- Va a tener que programar dispositivo por dispositivo, a fin de poder establecer uno como **TRANSMISOR** y el otro como **RECEPTOR**.
- Tendrá que conectar dos dispositivos a la vez, usando los conversores TTL y tener dos ventanas de Arduino (se recomienda que sea en dos laptops diferentes).



(Consultar al docente sobre cualquier duda en implementación).

3.1. Instrucciones:

1. Verificar la correcta conexión enviando el comando AT (o ATZ)
2. Desactivar LoRaWAN y activar modo P2P con el comando:

AT+NWM=0

3. Configurar los parámetros de enlace P2P

AT+P2P=923000000:7:125:0:10:14

- 923000000: frecuencia en Hz

Nota: A fin de aislar las comunicaciones, su grupo usará la banda de frecuencia:

923X00000

Donde X es su número de grupo. Si su grupo es el 3, usted usará la frecuencia 923300000.

- 7: SF7 (Spreading Factor alto = largo alcance)

- 125: ancho de banda (kHz)
- 0: coding rate 4/5
- 10: longitud del preámbulo.
- 14: potencia de transmisión (dBm)

4. RECEPTOR

- Después de configurar el modo P2P, activa la recepción continua:

AT+PRECV=65534

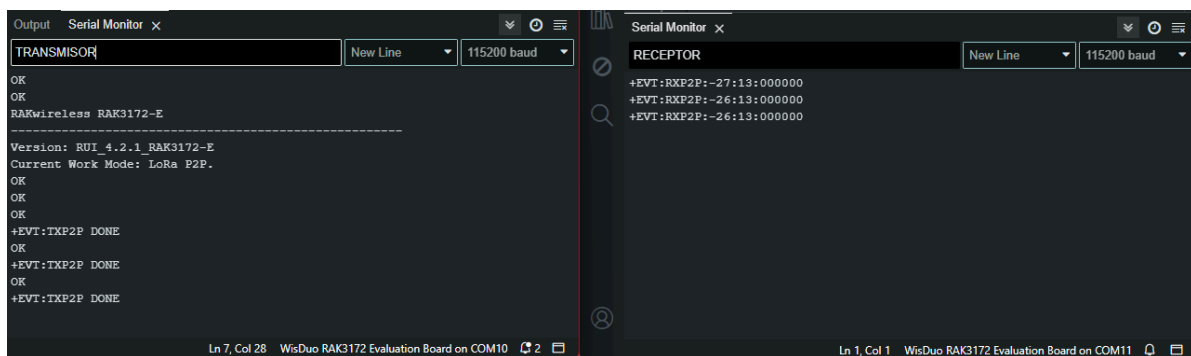
5. TRANSMISOR

- Después de configurar el modo P2P, activa la transmisión:

AT+PRECV=0

6. Envío de mensaje:

AT+PSEND:<Mensaje en Hexadecimal>



Checkpoint 03

4. Implementación con Arduino y sensor

1. Elija uno de los dispositivos para que sea conectado con el Arduino MEGA. La comunicación deberá hacerla a través del puerto **Serial1 del Arduino y el Serial2 del RAK.**
2. Debe de hacerlo en laptops diferentes.
3. Escoger un sensor cualquiera, que sea medido con el Arduino MEGA y enviar el mensaje usando LoRa al dispositivo RAK Receptor.

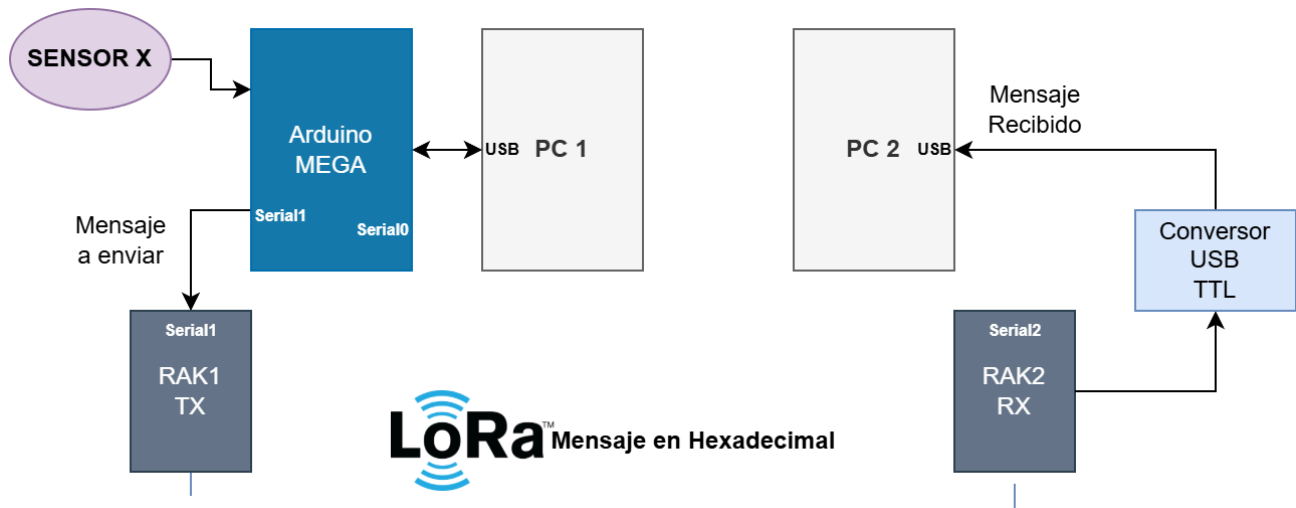


Figura 4. Diagrama de comunicación

4. Implemente un código que haga el envío de un mensaje LoRa (Recuerde que debe estar codificado en hexadecimal). Debe ser visible en el monitor serial conectado al dispositivo receptor. En la última sección se deja un ejemplo de código.

Checkpoint 04

Nota. Recuerde realizar la conversión de datos a Hexadecimal.

ANEXOS

5. LoRa

Es una tecnología de modulación patentada por Semtech que permite la transmisión de datos de largo alcance (varios kilómetros) utilizando frecuencias de radio no licenciadas. LoRa se centra en la capa física de la comunicación inalámbrica y proporciona una forma eficiente de transmitir datos en entornos con interferencias y con un consumo de energía relativamente bajo.

5.1. Características

- **Largo alcance:** Puede alcanzar distancias de varios kilómetros a decenas de kilómetros, dependiendo de las condiciones y la potencia de la señal.
- **Baja potencia:** Las transmisiones LoRa consumen muy poca energía, lo que las hace ideales para dispositivos alimentados por baterías con una larga vida útil.
- **Penetración de obstáculos:** La capacidad de penetración a través de edificios y otros obstáculos es una ventaja clave en entornos urbanos.
- **Ancho de banda estrecho:** Utiliza un ancho de banda de espectro estrecho, lo que lo hace adecuado para aplicaciones de baja velocidad de datos.

6. Dispositivos RAK

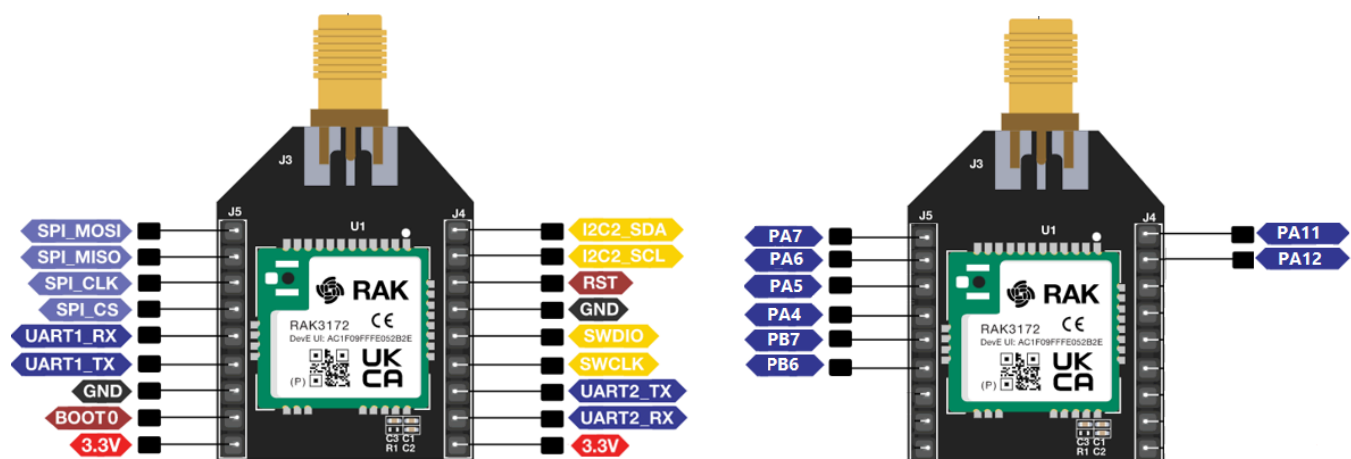


Figura 7. Pinout de dispositivo RAK3172-SiP.

Más información: [RAK3172S](#)

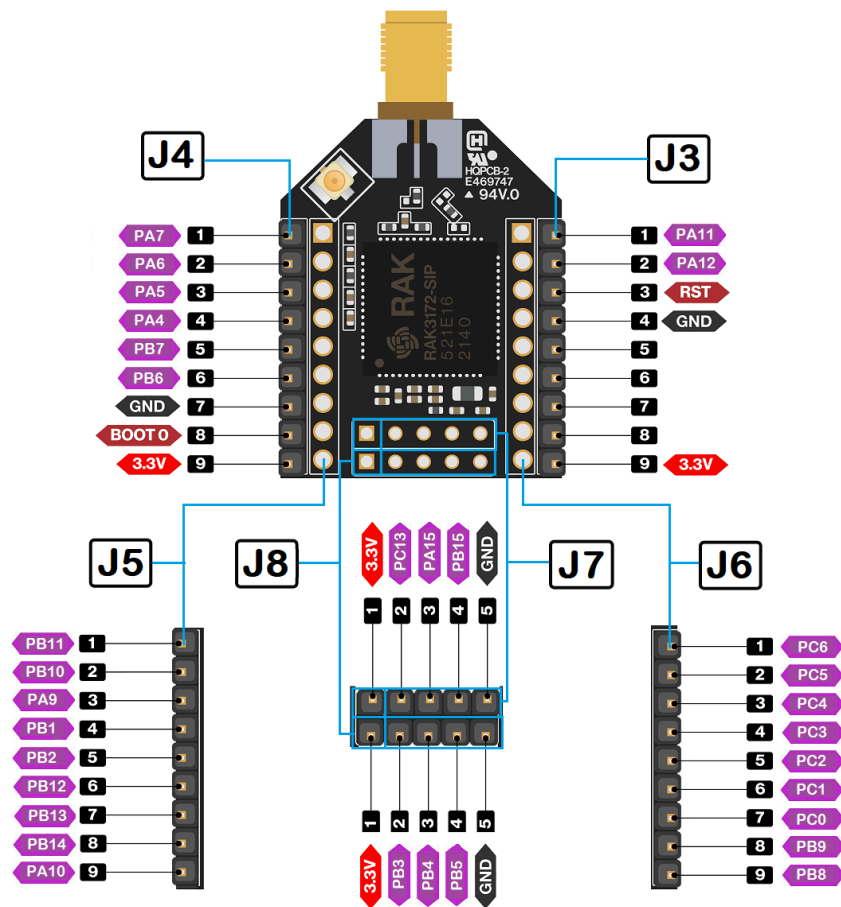


Figura 8. Pinout de dispositivo RAK3172-SiP.

Más información: [RAK3172S-SiP](#)

7. Conexión de dispositivos RAK con Conversor TTL

Se debe conectar los dispositivos RAK a través de comunicación UART, usando un conversor USB a TTL.

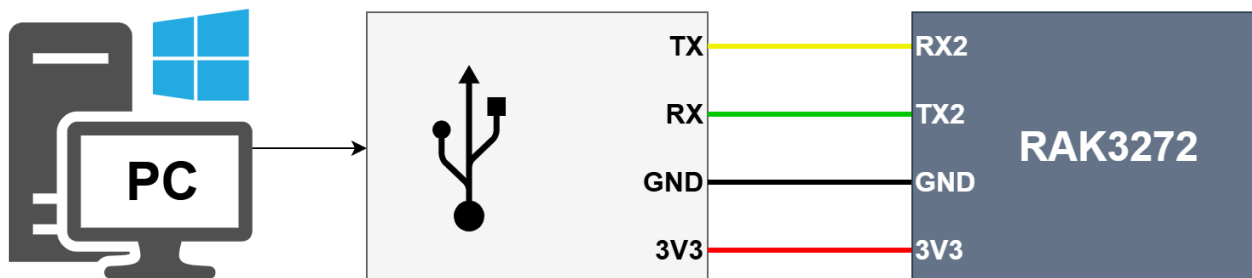
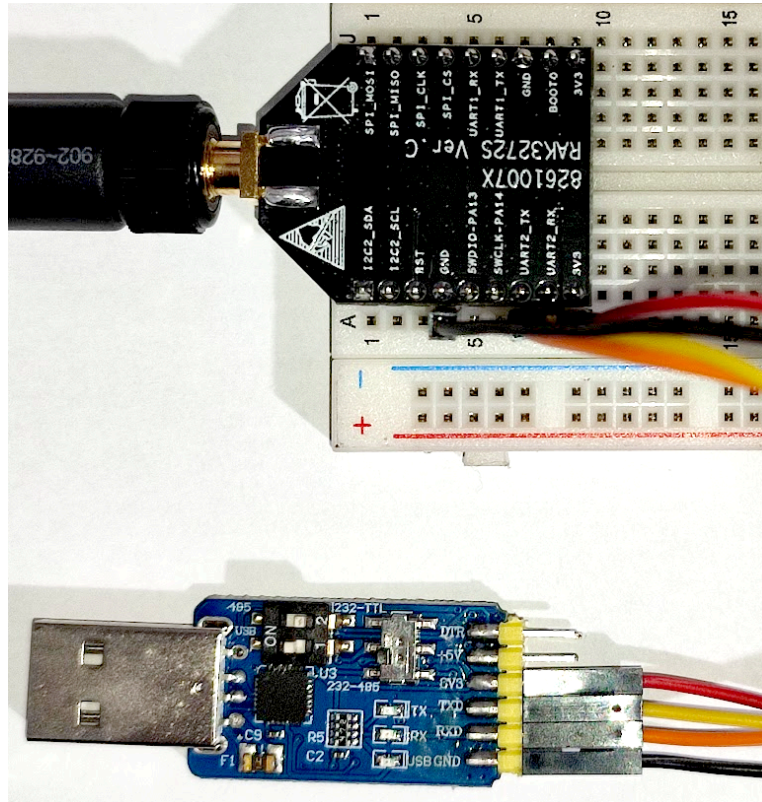
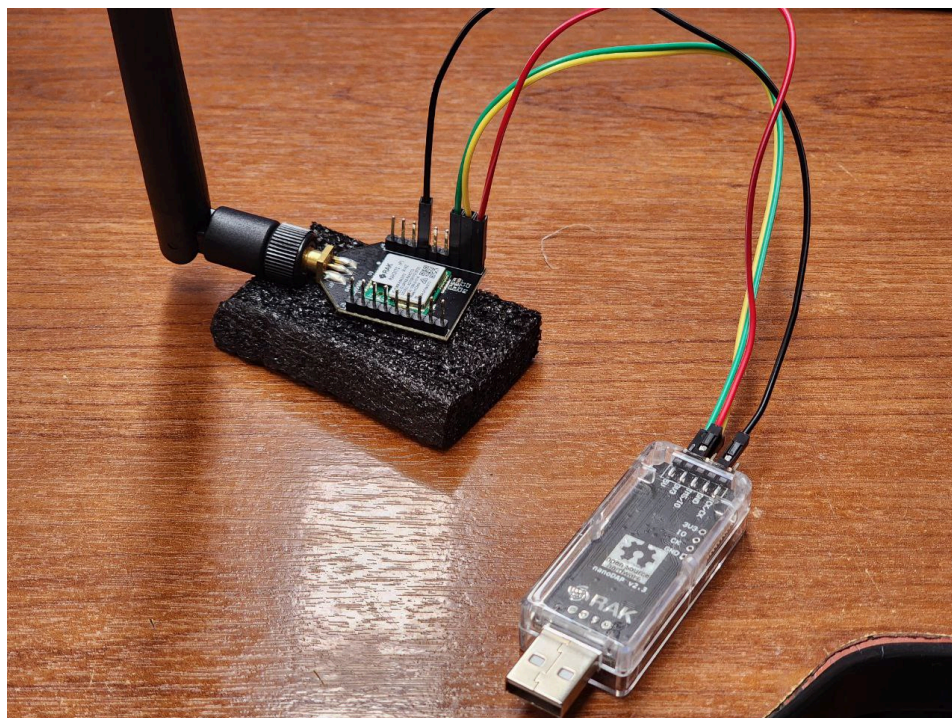


Figura 2. Esquema de conexión de PC a RAK usando conversor TTL.



Ejemplo de conexión con RAK3172S a través de conversor TTL 6 en 1.

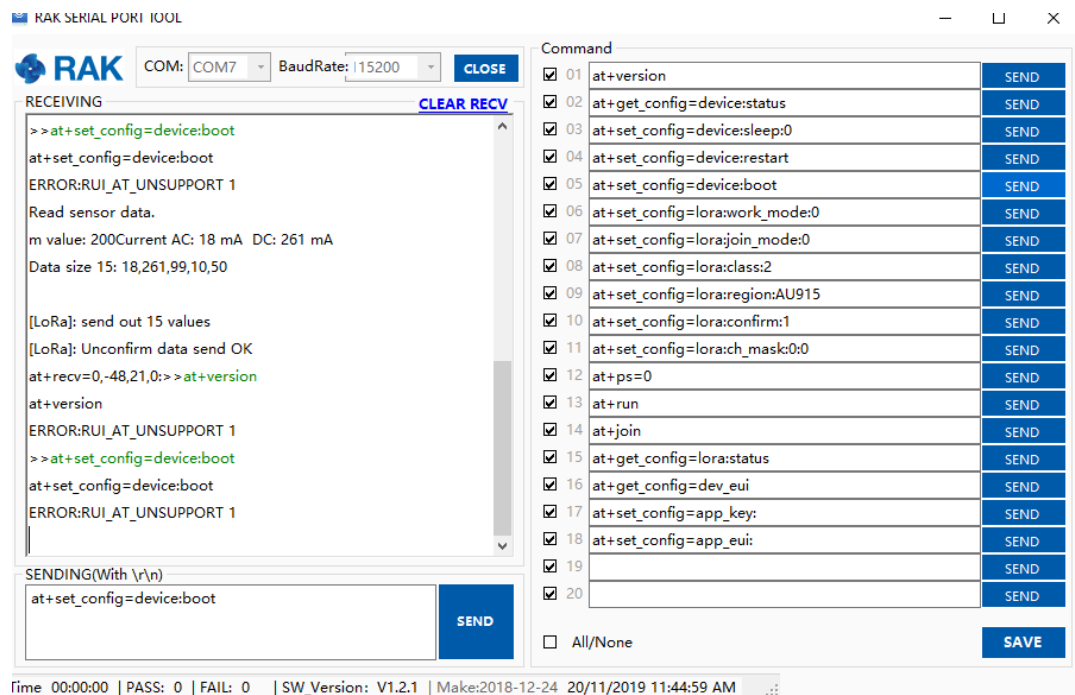


Ejemplo de conexión con RAK3172S a través de RAK DAP.

8. RAK SERIAL PORT TOOL

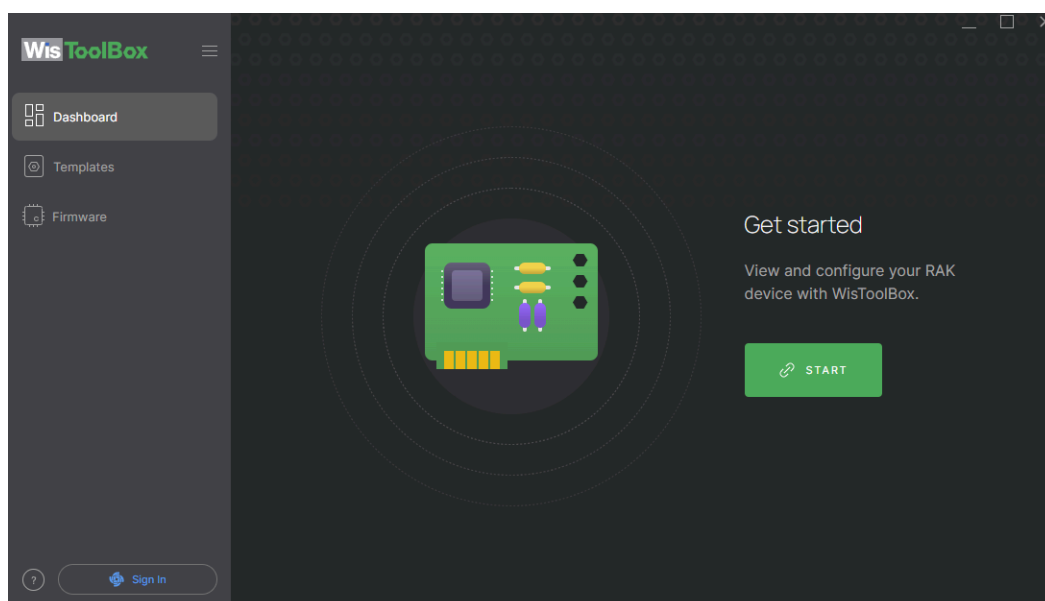
Nota: Es posible que pida instalar .NET al ejecutar el programa.

.NET Framework 3.5 (includes .NET 2.0 and 3.0)

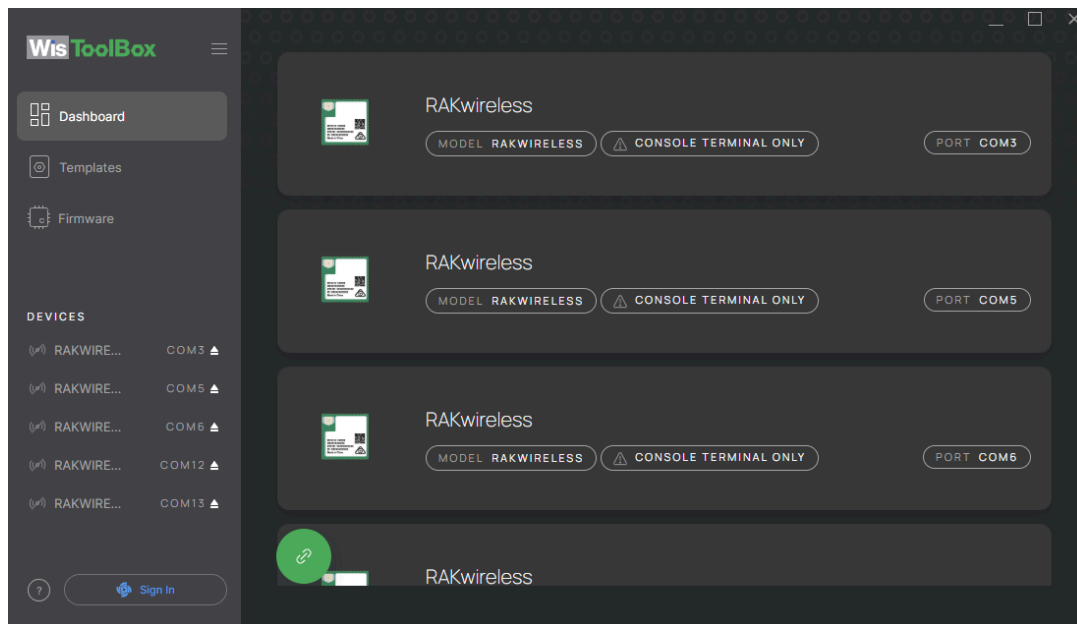


Ventana de RAK Serial Port Tool.

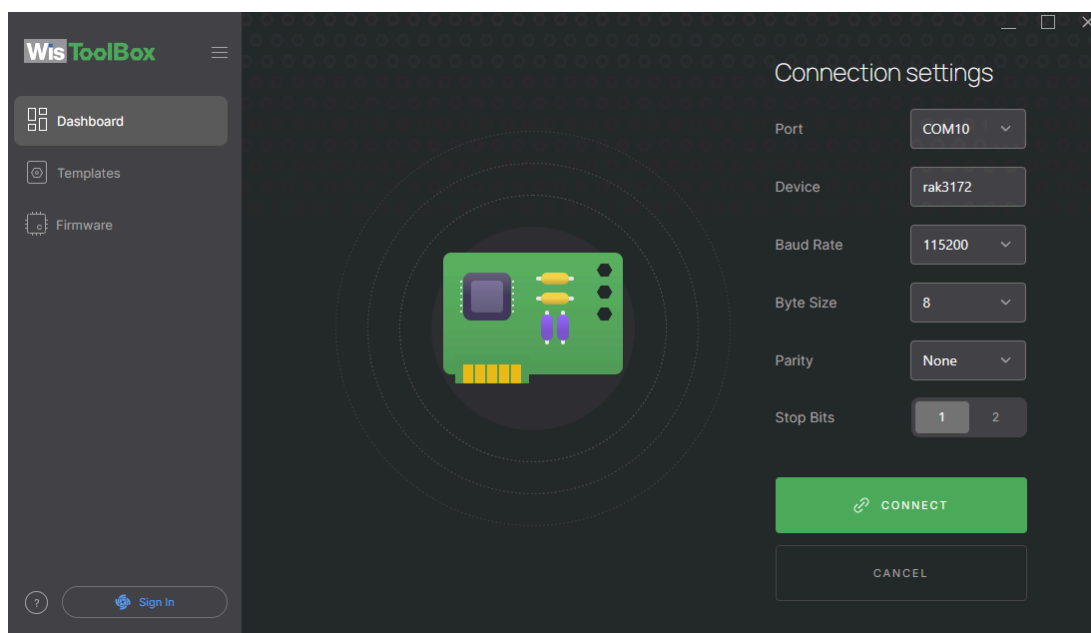
9. RAK WisToolBox



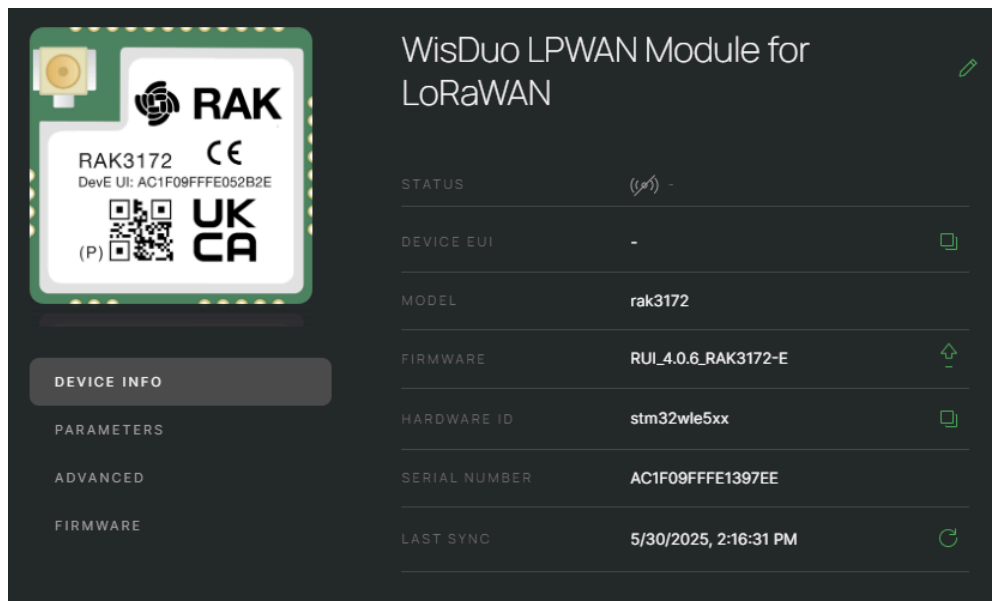
Software WisToolBox



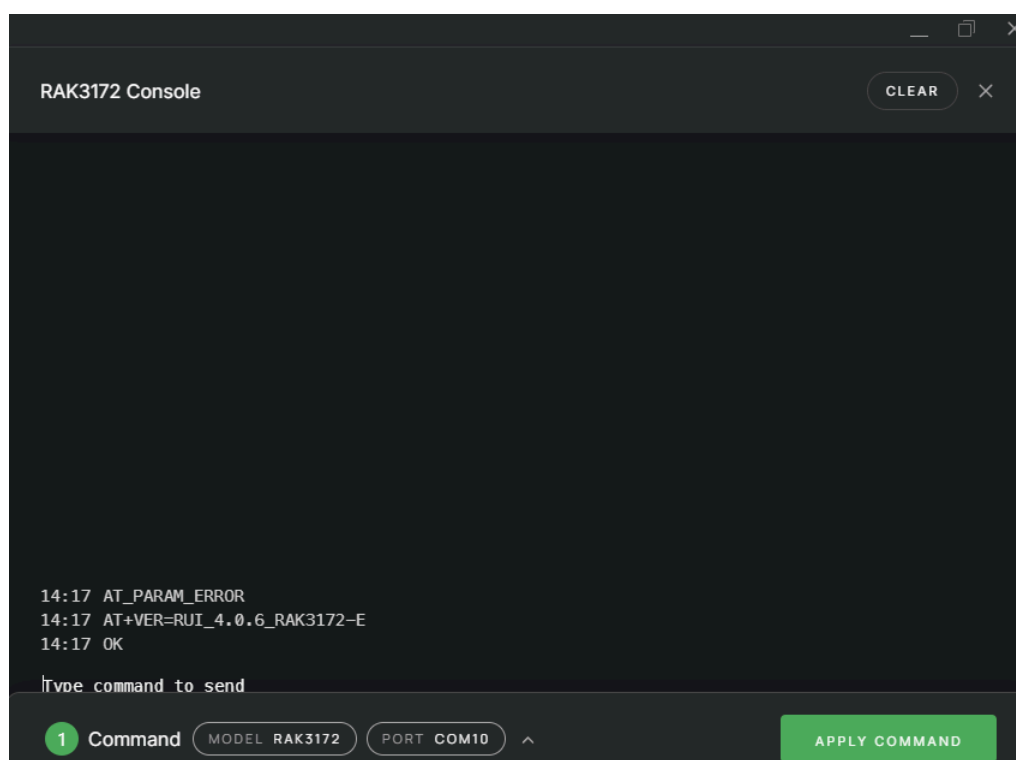
Identificación automática de dispositivos conectados.



Selección Manual de dispositivo (Usar los mismos parámetros).



Información del dispositivo.

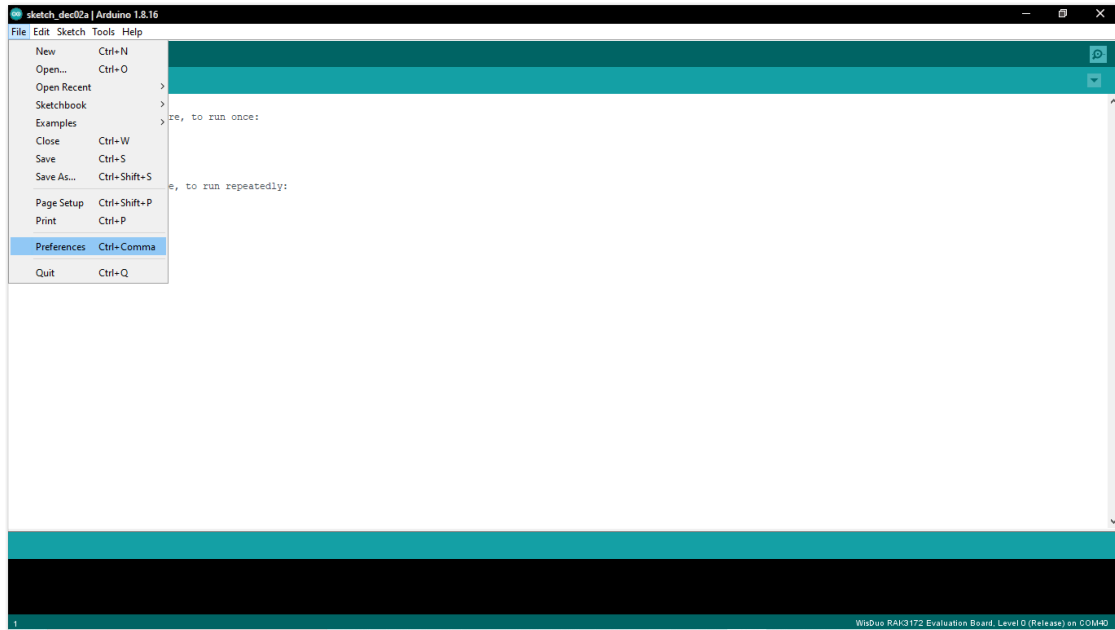


Prueba de comandos por consola.

10. Paquete de soporte de la placa RAK3272S RUI3 en Arduino IDE

También se tiene como opción la conexión con el dispositivo RAK usando el IDE de Arduino, similar al ESP32.

1. Open Arduino IDE and go to File > Preferences.

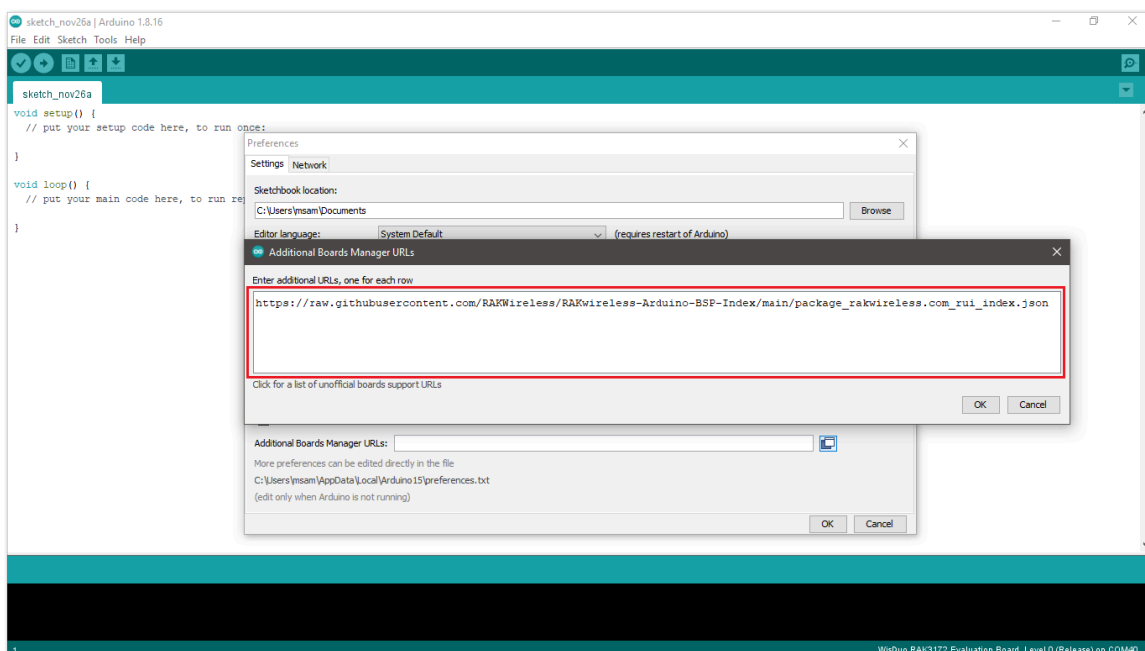


2. To add the RAK3272S to your Arduino Boards list, edit the Additional Board Manager URLs and click the icon, as shown in Figure 5.

3. Copy the URL

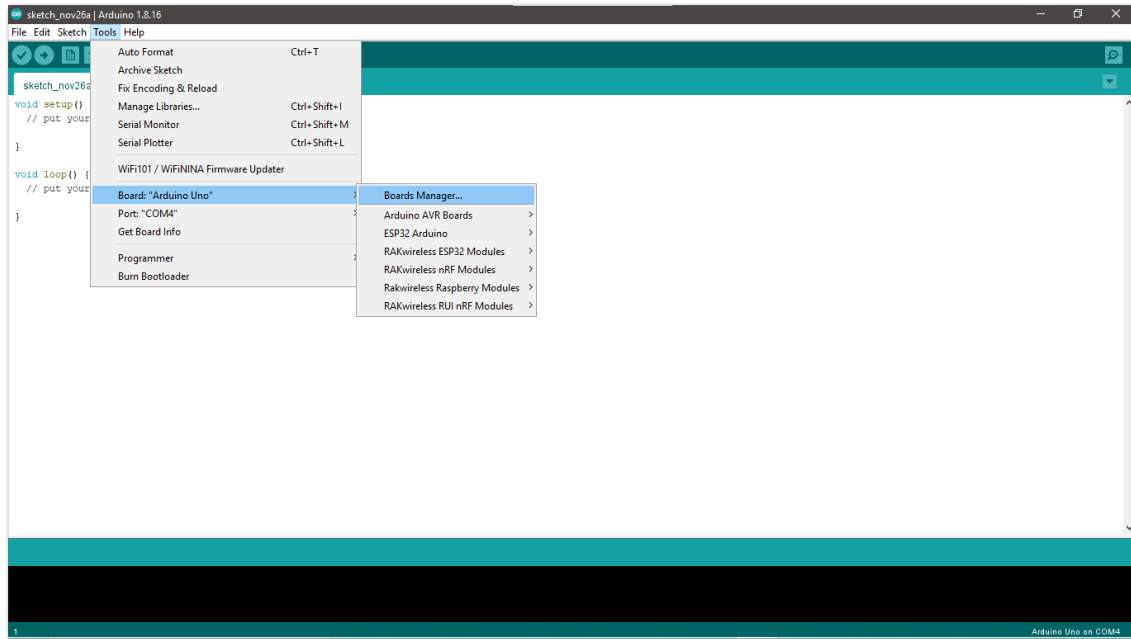
https://raw.githubusercontent.com/RAKWireless/RAKwireless-Arduino-BSP-Index/main/package_rakwireless_com_rui_index.json

and paste it on the field. If there are other URLs already there, just add them on the next line. After adding the URL, click OK.

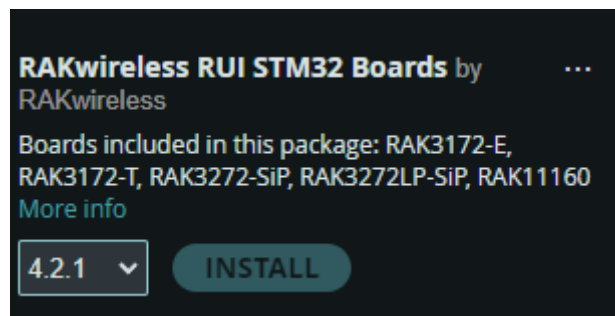


4. Restart the Arduino IDE.

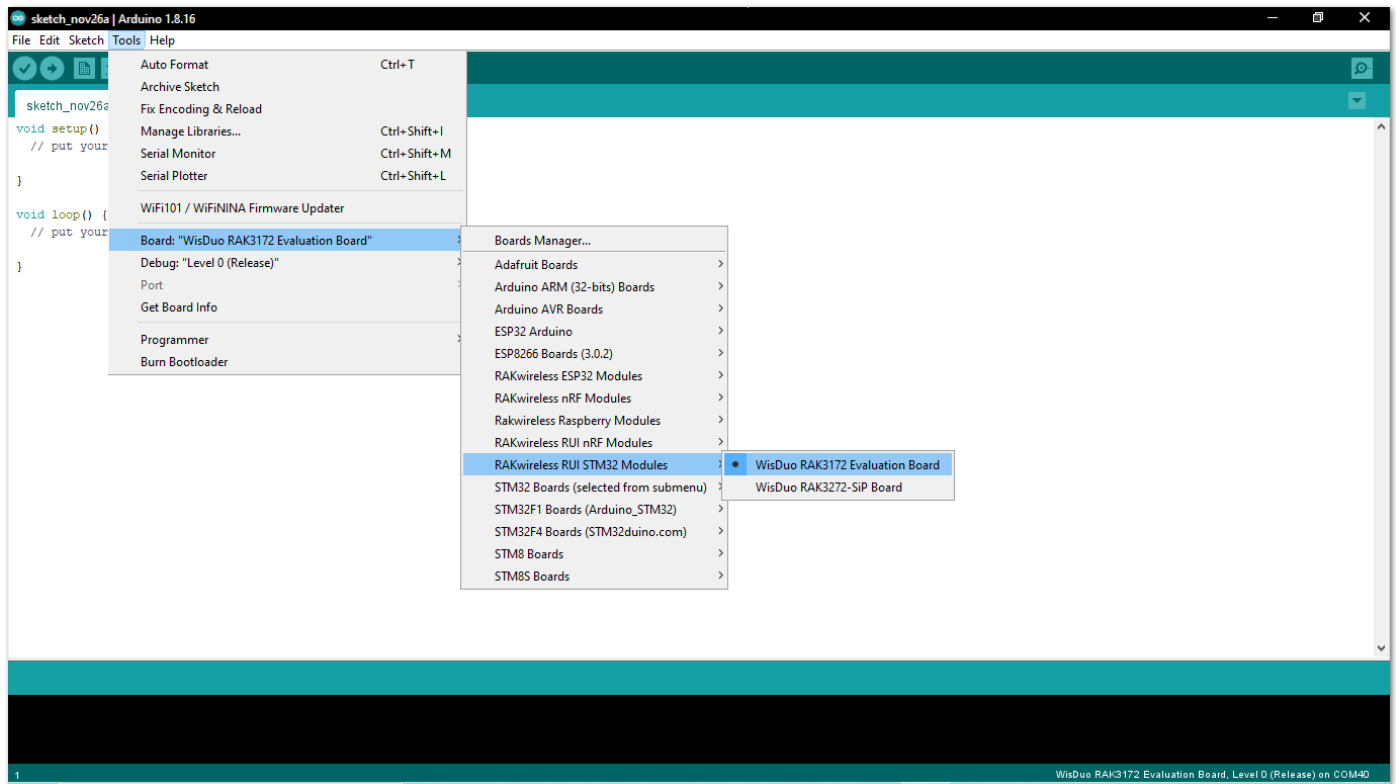
5. Open the Boards Manager from Tools Menu.



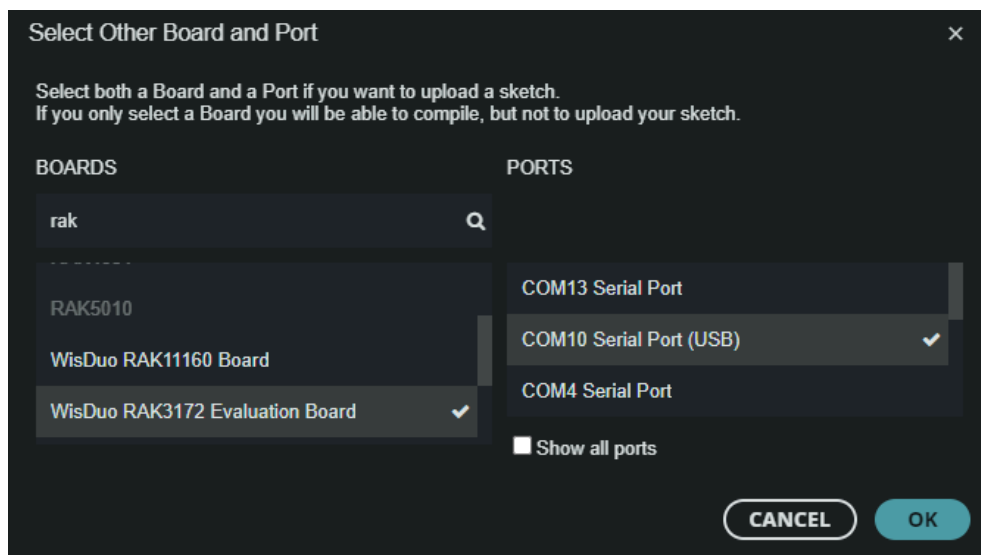
6. Type RAK in the search bar, as shown in Figure 8. This will display the available RAKwireless module boards that can be added to your Arduino board list. Select and install the latest version of the RAKwireless RUI STM32 Boards.



7. Once the BSP is installed, select Tools > Boards Manager > RAKWireless RUI STM32 Modules > WisDuo RAK3172 Evaluation Board. The RAK3272S Breakout Board uses RAK3172 WisDuo module.

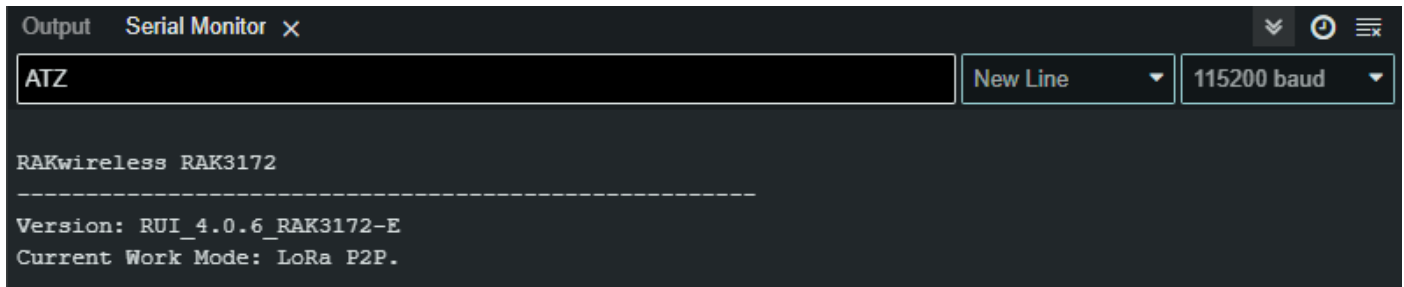


8. Select the board on the serial (COM) port where it is connected and configure the device as RAK3172.



9. Open the serial monitor and try sending an AT command (e.g. ATZ)

Nota: Por defecto el baudrate de estos dispositivos es de 115200.



```
Output  Serial Monitor x
ATZ
New Line 115200 baud

RAKwireless RAK3172
-----
Version: RUI_4.0.6_RAK3172-E
Current Work Mode: LoRa P2P.
```

11. Comandos AT (RUI v3)

Nota: Toda la información extendida acerca de los comandos AT se encuentra en el [repositorio](#) de RAKWireless.

La RAK3272S Breakout Board está desarrollada en torno al chip STM32WLE5CC y está diseñada para simplificar la comunicación LoRaWAN y LoRa punto a punto (P2P).

Para integrar la tecnología LoRa en sus proyectos, la RAK3272S proporciona una interfaz de comunicación UART fácil de usar, que le permite enviar comandos AT. Estos comandos le permiten configurar parámetros para la comunicación LoRa P2P y LoRaWAN. Además, se puede utilizar cualquier microcontrolador con una interfaz UART para controlar la placa RAK3272S.

La interfaz de comunicación serie UART está disponible en UART2 (también identificado como el puerto LPUART1), accesible a través del Pin 7 (TX2) y el Pin 8 (RX2). La configuración por defecto de la comunicación UART2 es 115200 baudios, 8 bits de datos, sin paridad y 1 bit de parada (8-N-1). Las actualizaciones de firmware también se pueden realizar a través de este puerto. Para más detalles sobre la distribución de pines y un esquema de aplicación de referencia, consulte la [hoja de datos](#) de la placa de interconexión RAK3272S.

11.1. RUI3 Formato de comandos AT

Los comandos AT tienen el formato estándar AT+XXX, donde XXX denota el comando. Hay cuatro comportamientos de comando disponibles:

- **AT+XXX?** : proporciona una breve descripción del comando dado, por ejemplo, AT+DEVEUI?
- **AT+XXX** : se utiliza para ejecutar un comando, como AT+JOIN.
- **AT+XXX=?** : se utiliza para obtener el valor de un comando dado, por ejemplo, AT+CFS=?
- **AT+XXX=<valor>** : se utiliza para proporcionar un valor a un comando, por ejemplo, AT+CFM=1.

El formato de salida es el siguiente:

```
AT+XXX=<value><CR><LF>
<CR><LF><Status><CR><LF>
```

<CR> significa «carriage return» y <LF> significa «line feed».

La salida <value><CR><LF> se devuelve siempre que se ejecutan los comandos «help AT+XXX?» u «get AT+XXX=?».

Los posibles códigos de estado son:

- **OK:** el comando se ejecuta correctamente sin errores.
- **AT_ERROR:** error genérico.
- **AT_PARAM_ERROR:** un parámetro del comando es incorrecto.
- **AT_BUSY_ERROR:** la red LoRa está ocupada, por lo que el comando no se ha completado.
- **AT_TEST_PARAM_OVERFLOW:** el parámetro es demasiado largo.
- **AT_NO_CLASSB_ENABLE:** el nodo final aún no se ha conectado en clase B.
- **AT_NO_NETWORK_JOINED:** la red LoRa aún no se ha conectado.
- **AT_RX_ERROR:** detección de error durante la recepción del comando.

```
14:17 AT_PARAM_ERROR
14:17 AT+VER=RUI_4.0.6_RAK3172-E
14:17 OK
```

Ejemplo de ejecución de comandos.

Comando	Valor de Entrada	Valor de Respuesta	Descripción
AT	-	OK	Comando que permite verificar que la comunicación con el dispositivo es funcional.
ATZ	-	OK	Reinicia el módulo y brinda la información del dispositivo.
ATR	-	OK	Restaura todos los parámetros a los valores iniciales por defecto.
AT+SN=?	-	<1-18char>	Numero serial del dispositivo.
AT+VER=?	-	AT+VER=<ver>	Versión del firmware del dispositivo.
AT+SLEEP	-	OK	Activa el modo sleep.
AT+SLEEP=<val>	(Int) Tiempo en ms	OK	Activa el modo sleep durante el tiempo especificado en ms.
AT+BAUD=?		AT+BAUD=<bauds>	Obtiene la velocidad en baudios del puerto serie.
AT+BAUD=<val>	(Int) Velocidad en baudios.	OK	Configurar la velocidad en baudios del puerto serie.

Comando	Valor de Entrada	Valor de Respuesta	Descripción
AT+NWM=<0,1>	1 Bit	OK	Comando que configura el modo de red empleado por el dispositivo. 0: LoRa P2P. 1: LoRaWAN.
AT+P2P=<f>:<sf>:<bw>:<cr>:<pl>:<p>	Varios valores.	OK	Comando de configuración del modo P2P, donde: f: Frecuencia de enlace. Sf: Spreading Factor. bw: Ancho de banda en kHz. cr: Tasa de codificación. pl: Longitud del preámbulo. p: Potencia en dBm.
AT+PRECV=<t>	Tiempo ms	OK	Comando para establecer la duración de la recepción en ms. Donde hay casos especiales: 0: El dispositivo funcionará únicamente como TX. 65535: Funcionará como RX hasta que reciba un mensaje, y luego cambia a TX (0). 65534: Funciona únicamente como RX, hasta volver a configurarlo.
AT+PSEND=<payload>	Mensaje Hexadecimal	OK	Comando para enviar un mensaje <payload> en formato hexadecimal compuesto por hasta 256 caracteres hexadecimales.

Comandos RUI3 empleados en comunicación LoRa.

12. Código de Ejemplo Arduino + RAK

```
int num = 20;
String hexNum;
void setup() {
    // initialize both serial ports:
    Serial.begin(115200);
    Serial1.begin(115200);
    delay(1000);
    sendRAK("at+ver=?");
    delay(500);
    sendRAK("at+ver=?");
    delay(500);
    sendRAK("at+NWM=0");
    delay(500);
    sendRAK("at+P2P=923000000:7:125:0:10:14");
    delay(500);
    sendRAK("at+PRECV=0");
    delay(500);
}

void loop() {
    num = num + 1;
    hexNum = stringToHex(String(num));
    sendRAK("at+PSEND=" + String(hexNum));

    // Wait for a second before sending the next message
    delay(5000);
}

void sendRAK(String message) {
    // Define the string to be sent
    Serial.print("AT COMMAND SENT: ");
    Serial.println(message);

    // Send the string through Serial1
    Serial1.println(message);

    // Optionally, print the message to the Serial Monitor for debugging
    Serial.println("Sent: " + message);

    // Wait for a response
    while (Serial1.available() == 0) {
        // Do nothing and wait for data to become available
    }
}
```

```
}

// Read the response from Serial1
String response = "";
while (Serial1.available() > 0) {
    response += (char)Serial1.read();
}

// Print the response to the Serial Monitor
Serial.println("Received: " + response);
}

String stringToHex(String str) {
    String hexString = "";
    for (int i = 0; i < str.length(); i++) {
        char hex[3]; // Two characters for the hex code plus the null terminator
        sprintf(hex, "%02X", str[i]);
        hexString += hex;
    }
    return hexString;
}
```